

Teranga Shield

Filtrer, analyser et expliquer les appels et les SMS
frauduleux — entièrement sur l'appareil, hors ligne.

21

COMMITTS DE
DÉVELOPPEMENT

3

RÔLES SYSTÈME PAR
DÉFAUT

FR·EN·RU

LANGUES SUPPORTÉES

100%

TRAITEMENT HORS
LIGNE

Une app Android qui se substitue au téléphone et aux messages par défaut pour repérer, en temps réel et sans connexion internet, les schémas classiques d'arnaque (faux conseiller bancaire, demande de code, urgence fabriquée, usurpation d'institution).

Contrairement à un antivirus ou un bloqueur de spam générique, Teranga Shield explique *pourquoi* un appel ou un SMS est suspect — score de risque, phrases déclenchantes, catégorie d'arnaque reconnue — plutôt que de se contenter d'un blocage silencieux. Toute l'analyse (voix et texte) tourne localement sur l'appareil : rien n'est envoyé sur un serveur.

■ Filtrage des appels • Numéro vérifié avant la sonnerie • Alerte si signalé par la communauté • Écoute + transcription en direct si haut-parleur activé	■ Filtrage des SMS • Analyse de chaque message reçu • Détection de lien suspect, désactivé automatiquement • Score + raison affichés	■ Confidentialité • 100% local, aucun envoi réseau • Consentement micro séparé et explicite • Transcript effacé, jamais stocké en clair si risque élevé
---	---	--

Architecture technique

COUCHE	CHOIX	POURQUOI
UI	Kotlin + Jetpack Compose, MVVM	Déclaratif, réactif (StateFlow), standard moderne Android
Injection de dépendances	ServiceLocator manuel (pas de Hilt)	Build plus léger et plus rapide, cohérent avec la contrainte de sobriété du produit
Persistance	Room (SQLite) + DataStore	Historique appels/SMS/signalements en local, préférences utilisateur
Téléphonie	android.telecom — CallScreeningService + InCallService	Filtrage pré-sonnerie et rôle téléphone par défaut complet
SMS	Rôle SMS par défaut (SmsDeliverReceiver, etc.)	Seule voie pour intercepter et analyser chaque message entrant
Reconnaissance vocale	SpeechRecognizer embarqué Android (API 31+)	Garanti 100% hors-ligne, évite d'embarquer un modèle Whisper volumineux
Détection	Moteur NLU par correspondance de motifs pondérés (mock, interface prête pour un vrai modèle ML)	Résultats démontrables immédiatement, remplaçable sans toucher au reste de l'app

FAIT

Rôles système par défaut — Appels (CallScreeningService), Téléphone (InCallService / dialer complet), SMS

L'app sonne, décroche, affiche le clavier et gère l'historique comme un vrai téléphone par défaut.

FAIT

Historique unifié — appels entrants/sortants, contacts connus, SMS reçus/envoyés

Import de l'historique existant à l'activation, plus tout nouvel événement.

FAIT

Analyse vocale en direct pendant un appel — reconnaissance vocale embarquée + moteur de score

Se déclenche à l'activation du haut-parleur ; alerte + vibration + notification au contact de confiance si risque élevé.

FAIT

Analyse de chaque SMS — y compris venant d'un contact enregistré

Un compte de contact compromis demandant un code est un schéma d'arnaque réel ; plus aucune exemption automatique.

FAIT

Signalement communautaire (local) — base Bloom filter des numéros signalés, historique de ses propres signalements

Le partage réellement communautaire (serveur central) reste hors périmètre — voir Limites.

FAIT

Multilingue — Français / Anglais / Russe, y compris les jeux de motifs de détection

La langue choisie détermine à la fois l'interface et le vocabulaire d'arnaque reconnu.

FAIT

Consentements séparés et explicites — rôles par défaut d'un côté, micro de l'autre

Corrigé récemment : le bouton de consentement exigeait auparavant une seule case, laissant le micro bloqué sans le vouloir.

FAIT

Identité visuelle "Modernist" — police Archivo, accent unique, angles à zéro

Appliquée à l'ensemble de l'app (accueil, appels, messages, contacts, réglages) à partir d'une maquette de design dédiée.

EN COURS

Fiabilisation sur appareil réel — série de correctifs post-tests terrain

Numéro/bouton raccrocher invisibles, réception SMS non garantie si le rôle système est repris en arrière-plan : filets de sécurité ajoutés, validation utilisateur en cours.

PRÉVU

Vrai modèle de détection ML (NLU + classification vocale)

L'architecture est prête (interfaces RiskAnalysisEngine / VoiceClassifierEngine) ; l'implémentation actuelle est un moteur à motifs pondérés, pas un modèle entraîné.

PRÉVU

Base de signalements réellement partagée entre utilisateurs

Nécessite un serveur/backend — hors périmètre d'une app strictement "tout sur l'appareil".

Où en est le projet, en une phrase : le parcours complet fonctionne de bout en bout sur un vrai téléphone (appel, SMS, alerte, historique), le cœur du travail actuel porte sur la robustesse face aux comportements variables des appareils Android (permissions, rôles système, reconnaissance vocale hors-ligne selon le fabricant).

Détection

- Moteur actuel = correspondance de motifs pondérés, pas un modèle ML entraîné sur des données réelles d'arnaque
- La reconnaissance vocale hors-ligne d'Android dépend du fabricant et du pack de langue installé sur l'appareil — pas garanti uniforme sur tous les téléphones

Plateforme

- Android uniquement (aucune version iOS envisagée à ce stade)
- Les rôles "app par défaut" (SMS, appels, téléphone) dépendent de l'accord explicite de l'utilisateur, révocable à tout moment par le système
- Testé essentiellement en conditions réelles sur un seul modèle d'appareil à ce stade

Signalements

- Pas de base communautaire partagée en temps réel (nécessiterait un serveur, hors du principe "tout local")
- Chaque utilisateur ne voit pour l'instant que ses propres signalements

Environnement de build

- Développement et tests effectués via intégration continue (GitHub Actions), sans environnement Android Studio local fonctionnel
- Cycle de correction un peu plus lent qu'en local (~3 minutes par vérification)

Prochaines étapes proposées

■ Court terme

- Valider les correctifs récents sur plusieurs appareils réels
- Élargir la couverture de test (marques Android variées)
- Petits ajustages d'ergonomie selon retours utilisateurs

■ Moyen terme

- Remplacer le moteur de détection par un vrai modèle NLU entraîné
- Étudier un backend léger pour le partage communautaire des signalements
- Tests utilisateurs élargis (public non technophile, incl. seniors)

Kotlin

Jetpack Compose

Room

DataStore

android.telecom

SpeechRecognizer on-device

GitHub Actions CI

100% offline